A Language for Efficient Event Log Repair

Sareh Sadeghianasl[1 [0000−0002−0338−958X]], Remco Dijkman[2 [0000−0003−4083−0036]], Robert Andrews[1 [0000−0001−7743−5772]], and Alistair Barros[1 [0000−0001−8980−6841]]

[1] Queensland University of Technology, Brisbane, Australia
{s.sadeghianasl,r.andrews,alistair.barros}@qut.edu.au
[2] Eindhoven University of Technology, Eindhoven, The Netherlands
R.M.Dijkman@tue.nl

Abstract. Data quality is a fundamental desirable feature of any event log to enable reliable *process mining*. Unfortunately, real-life event logs often suffer from data quality issues, many of which have been codified in the form of a pattern collection. What remains missing is a user-friendly and computationally efficient approach to repairing these issues. To fill this gap, this paper introduces a language for specifying repair expressions and a tool that supports event log repair using these expressions. The language is formally grounded in relational algebra and extends it with a new generalised group-by operator, which can be used in combination with an earlier defined direct-follows operator and classical operators to specify event log repair expressions. Compared to standard database operators, these dedicated extensions yield repair expressions that are easier to construct and understand and computationally efficient. A pattern-based evaluation shows lower time complexity and higher suitability. An evaluation with our prototype further demonstrates at least 50 times faster runtime on a real-world event log, compared to expressions formulated with standard database operators.

Keywords: Repair operator · Event logs · Data quality.

1 Introduction

The analysis of process event logs is studied through the field of process mining in support of process improvement and intelligence [1]. As in any area of data science, data quality is paramount when analysing process event logs and drawing valid conclusions. Nonetheless, process event logs frequently suffer from common data quality problems, such as missing data, in addition to quality issues that are specific to event logs, such as inaccurate timestamps. The presence of data quality issues specific to event logs has led to the identification of imperfection patterns [30,28], which describe the nature of the problems, ways of detecting them, and potential remedies. While the patterns systematise data quality management for process event logs, their implementation (i.e., automated detection and repair) is left to ad-hoc handling. This often takes place as part of data

preprocessing in process mining projects, involving considerable lead times and cost and over-reliance on experience and expertise (see e.g. [35]).

To overcome these issues, expressions for event log repair are needed that are easy to use, re-use, and adapt. These expressions should be *suitable* for use, meaning they must be easy to write and understand, and fit with existing operators. The expressions and operators should also be *computationally efficient*.

To fill this gap, this paper proposes a language for creating event log repair expressions. We specify the operators that make up the language using relational algebra and provide a practical implementation of these operators in the PraeclarusPDQ open-source framework[3], which is developed explicitly for event log quality management. To create a language that is suitable and computationally efficient, we introduce a new operator called *generalised group-by* and prove its consistency with classical relational algebra. This operator, along with the directly-follows operator, developed in an existing work [11], provides a basis for specifying and conducting event log repairs.

We evaluate the resulting language by showing how it can be used to repair common patterns of imperfection in event logs and comparing the repair expressions in our language to expressions that use only standard (SQL) database operators. We also refer to expressions that do not include the generalised group-by and the directly follows operators as '*vanilla*' expressions and to expressions that do as '*dedicated*' expressions. While vanilla database operators can of course be used to write event log repair expressions, our evaluation shows that dedicated repair expressions are simpler, making them easier to construct and understand, and have lower theoretical time complexity and better runtime performance on a real event log.

Against this background, the remainder of this paper is organised as follows. In Section 2 related work is discussed. Section 3 introduces relational algebra and Section 4 formally introduces the new generalised group-by operator. To show the expressive power of the repair algebra operators, in Section 5 we provide repair statements for various manifestations of the event log imperfection patterns. In Section 6, an evaluation is presented, which shows that repair expressions without the help of the newly introduced repair operator tend to be more complex in terms of their suitability, computational efficiency, and runtime performance. Section 7 concludes the paper.

2 Related Work

Data cleaning, a key step in data preparation, detects and removes errors to improve data quality. Studies [34,24] report data scientists spend $\approx$ 80% of their time preparing data. Gartner's 'Magic Quadrant for Data Quality Tools' valued the market at $1.61 billion in 2017 [5]. Ehrlinger and Wö [14] identified 667 tools for *data quality*, while Côté et al. [9] reviewed 101 ML-based cleaning approaches. Several contributions propose semantically defined operators. Weiss and Manolescu [33] present XClean, with 8 operators based on the XQuery

[3] Available at `https://github.com/praeclaruspdq/PraeclarusPDQ`.

Data Model [32]. Khedri et al. [19] use *information algebra* [21] and association rules [2] to formalise structured data cleaning. Núnez-Corrales et al. [25] outline algebraic transformations from an initial dataset D_0 to a final dataset D_F.

Process mining event logs often contain quality issues. Cheng and Kumar [4] train classifiers to detect noisy traces. Conforti et al. [7] reduce noise via automata capturing direct-follows dependencies, removing infrequent transitions. Dixit et al. [12] propose semi-automated repair of timestamp anomalies. Conforti et al. [8] address same-timestamp events. In [29], activity context, i.e. control flow, resource, time, and data attributes, is used to detect semantically identical activity labels. In [27,26], crowd-sourced domain knowledge is used to detect and repair same-meaning activities with gamification to enhance user engagement.

Relational algebra, popularised by E.F.Codd in 1970 [6] as a basis for database query languages, includes five operations on relations - permutation, selection, projection, join(s), and restriction. Various contributions [10,20,3,16] have introduced extensions to relational algebra for specific purposes. Relational algebra forms the theoretical basis for (relational) database query languages such as Structured Query Language (SQL). Various authors [18,15,23,31] have proposed extensions to SQL to improve its generality and ease of use. Relational databases, as a repository for organisational data, are relevant to process mining as the source of event data. In [13] the authors present the RXES (Relational XES) framework for storing event data in a relational database structure. In [36] a set of operators are designed on relational algebra, but these manipulate logs already in XES format. In [11], the authors define a relational algebraic operator to extract the 'directly follows' relation, a construct frequently used in process discovery algorithms, from a log stored in a relational database and prove properties for the operator which can be used for query optimisation.

3 Preliminaries

We will use relational algebra as a basis for defining our event log repair operators. We use the definitions inspired by Maier [22]. Let $\mathcal{C} \subseteq \Sigma^*$ be a set of *columns*. $R \subseteq \mathcal{C}, R \neq \varnothing$ is a *table*, where $Schema(R) = R$, and $Pop(R) \in \wp(R \to \Omega)$ is the *population* of R. Ω is a set of values, and $\wp$ is the power set. A *tuple* t in R is defined as $t \in Pop(R)$, i.e. $t\colon R \to \Omega$. $Dom\colon C \to \wp(\Omega)$ is a function that returns the domain or type of a column, i.e. its set of possible values. For each $t \in Pop(R)$ and $c \in R$, $t[c] \in Dom(c)$. A *database* is a set of tables with their population. Table 1a shows an example of a table R with $Schema(R) = \{c, a, time, r\}$ and $Pop(R) = \{\ldots, \{c \mapsto c_1, a \mapsto a_0, time \mapsto 0, r \mapsto r_0\}, \ldots\}$.

Relational algebra expressions make use of a number of well-known operators, which are informally defined as follows. Let R and S be two tables:

- the *union* $R \cup S$ is a table consisting of all tuples that belong to R or S.
- the *set difference* $R \backslash S$ is a table consisting of tuples in R but not in S.
- the *join* $R \bowtie S$ is a table consisting of tuples that combine tuples of R with tuples of S that have the same values for the columns $R \cap S$ that they have in common. If $R \cap S = \varnothing$ then $R \bowtie S$ is the *Cartesian product* of R and S.

cid	*act*	*time*	*res*
.	.	.	.
c_1	a_0	0	r_0
c_1	a_1	1	r_1
c_1	a_2	2	r_1
c_2	a_2	1	r_3
c_2	a_1	2	r_3
c_2	a_3	3	r_3
.	.	.	.

(a) A relation

act	*time*	*res*
.	.	.
a_1	1	r_1
a_1	2	r_3
.	.	.

(b) Applying operators

$\downarrow$ *cid*	$\downarrow$ *act*	$\downarrow$ *time*	$\downarrow$ *res*	$\uparrow$ *cis*	$\uparrow$ *act*	$\uparrow$ *time*	$\uparrow$ *res*
.	.	.	.	.	.	.	.
c_1	a_0	0	r_0	c_1	a_1	1	r_1
c_1	a_1	1	r_1	c_1	a_2	2	r_1
c_2	a_2	1	r_3	c_2	a_1	2	r_3
c_2	a_1	2	r_3	c_2	a_3	3	r_3
.	.	.	.	.	.	.	.

(c) Directly follows application

Table 1: Relational algebra examples.

- the *selection* $\sigma_F(R)$ of R using a selection formula F is a table that only includes tuples that satisfy the selection formula.
- the *projection* $\pi_S(R)$, where $S = \{s_1, \cdots, s_n\} \subseteq Schema(R)$, is a table that only includes a subset of columns of R.
- the *extended projection* operator $\pi_{q_1 : F_1, \cdots, q_n : F_n}(R)$, where $q_1, \cdots, q_n \in \mathcal{C}$ are different column names and $F_1, \cdots, F_n$ are formulas over $Schema(R)$, enables applying functions to tuples [16].

Table 1b shows an example in which the operators $\pi_{\{a,time,r\}}(\sigma_{a=a_1}(R))$ are applied to R from Table 1a.

In event logs, the directly follows relation plays an important role. For that reason, we introduce the directly follows relation as it is previously defined [11] into the relational algebra. Let R be a tabular event log with $Schema(R) = \{cid, time, p_1, p_2, \ldots, p_n\}$, where *cid* is the column with case identifiers, and *time* the column with completion timestamps. Applying the directly follows operator, denoted $>_{cid,time} R$ to the event log returns the relation of events that follow each other in some case: $\{\{\downarrow cid \mapsto t[cid], \downarrow time \mapsto t[time], \downarrow p_1 \mapsto t[p_1], \ldots, \uparrow cid \mapsto u[cid], \uparrow time \mapsto u[time], \uparrow p_1 \mapsto u[p_1], \ldots\}, t \in R, u \in R, t[cid] = u[cid], t[time] < u[time], \neg\exists v \in R\colon t[cid] = v[cid] \wedge t[time] < v[time] \wedge v[time] < u[time]\}$.

Table 1c illustrates the use of the directly follows operator. The table presents an event log with two cases c_1 and c_2 with events at time *time*. Each event represents that an activity a was performed by a resource r. The result is the table that contains all pairs of events that directly follow each other in some case. For example, the event that activity a_0 is performed in case c_1 is directly followed by the event that activity a_1 is performed in case c_1. We assume that a directly precedes operator $<_{c,time} R$ exists that is defined analogously.

4 Generalised Group-By $\mathcal{G}$

To define the event log expressions conveniently we define an additional operator: the generalised group-by operator. This operator is implemented together with the other operators in the open-source framework PraeclarusPDQ.

The regular group-by operator groups tuples in a table R that have the same values for a subset $p_1, p_2, \ldots, p_n$ of the columns of R. As a result, each tuple

has a multiset of values in the other columns to which functions such as 'count', 'sum', and 'average' can be applied to yield a new table with unique values again.

In this paper, we introduce a generalised version of the group-by operator, in which tuples are grouped by a - more general - grouping condition, rather than by the values that they share in certain columns. We introduce this operator to be able to write event log repair expressions that are easier to create and understand and computationally less expensive than when using vanilla relational algebra.

The generalised group-by operator is defined as follows. Let R be a tabular event log. Furthermore, let $GC(t_1, t_2) \subseteq Pop(R) \times Pop(R)$ be a 'grouping condition', such that two tuples that are related by the condition must be grouped. F_i are grouping functions defined over the grouped rows from $Pop(R)$.

Definition 1. *For a table R, an grouping condition GC, and grouping functions F_i, the generalised group-by operator*

$$_{GC(t_1,t_2)}\mathcal{G}_{p_1 \mapsto F_1, p_2 \mapsto F_2, \ldots, p_n \mapsto F_n} R$$

is defined as:

- $Schema(_{GC(t_1,t_2)}\mathcal{G}_{p_1 \mapsto F_1, p_2 \mapsto F_2, \ldots p_n \mapsto F_n} R) = \{p_1, p_2, \ldots, p_n\}$
- $Pop(_{GC(t_1,t_2)}\mathcal{G}_{p_1 \mapsto F_1, p_2 \mapsto F_2, \ldots p_n \mapsto F_n} R) =$
  $\left\{p_i \mapsto t(p_i) \text{ if } p_i \in Schema(R) \text{ else } \bot \mid i \in \{1,2,\ldots,n\}, t \in Pop(R), \nexists t' \in Pop(R) : t' \neq t, GC(t,t')\right\} \cup$
  $\left\{\{p_i \mapsto F_i(ts) \mid i \in \{1,\ldots,n\}\} \;\middle|\; ts \subseteq Pop(R), |ts| > 1, \forall u, v \in ts : GC(u,v), \nexists z \in R \setminus ts : \forall u' \in ts : GC(u', z)\right\}$

Note that the groups $ts \subseteq Pop(R) \wedge |ts| > 1$ that are created in the second part of the definition are equivalent to each other with respect to the grouping condition, i.e. they are reflexive ($\forall t \in ts : GC(t,t)$), symmetric ($\forall t_1, t_2 \in ts : GC(t_1,t_2) \Leftrightarrow GC(t_2,t_1)$) and transitive ($\forall t_1, t_2 \in ts : GC(t_1,t_2) \wedge GC(t_2,t_3) \Rightarrow GC(t_1,t_3)$). This trivially follows from the fact that the groups are constructed such that $\forall u, v \in ts : GC(u,v)$.

Table 2 shows an example of the application of the generalised group-by operator. This table contains an event log with cases identified by column *cid*. On these cases, activities are performed with labels given in column *act*. The activities are completed at time *time* by resource *res*. In this table we want to group activities with labels a_1, a_2, a_3 that belong to the same case into a single activity, if those activities all happen within a time δ of each other. After grouping the activities, the activity label of grouped activities should become a_{new}, the completion time should become the time of completion of the last activity (i.e., the maximum time of the grouped activities), and the resource should become any one of the resources of the grouped activities, because we assume that the activities were performed by the same resource. We define a convenience function any_c, which for a table R returns any value from the column c. Analogously, we define functions max_c, min_c, sum_c, etc. The grouping expression that gives the desired result is $_{GC(t_1,t_2)}\mathcal{G}_{cid \mapsto any_{cid}, act \mapsto anew, time \mapsto max_{time}, res \mapsto any_{res}} R$, where the function $anew(a) = a_{new}$ is defined to simply return the desired label, a_{new}, for grouped activities, and $GC(t_1,t_2) = (t_1[cid] = t_2[cid]) \wedge t_1[act] \in \{a_1, a_2, a_3\} \wedge t_2[act] \in \{a_1, a_2, a_3\} \wedge |t_1[time] - t_2[time]| \leq \delta$ to group activities as described above.

This expression can be equivalently represented using vanilla relational algebra operators as we will show in Section 6. In general, for each generalised group-by expression, there is an equivalent expression using vanilla relational algebra operators. However, because the generalised group-by operator is a singular operator, it is easier to create and understand expressions that use it, compared to equivalent expressions represented using vanilla relational algebra operators. In addition, it is possible to create a more computationally efficient algorithm for its evaluation. These properties will be validated in the evaluation.

Proposition 1. *For each generalised group-by expression*

$$_{GC(t_1,t_2)}\mathcal{G}_{p_1 \mapsto F_1, p_2 \mapsto F_2, \ldots, p_n \mapsto F_n} R$$

there is an algorithm that has equivalent results, but only uses vanilla relational algebra operators.

Proof. The equivalent relational algebra algorithm is

$result = \emptyset$
for $t \in Pop(R)$:
  $R' = \sigma_{GC(t,\cdot)} R$
  if $R' = t$:
    $\{q_1, \ldots, q_m\} = \{p_1, \ldots, p_n\} \cup Schema(R)$
    $\{s_1, \ldots, s_k\} = \{p_1, \ldots, p_n\} \setminus Schema(R)$
    $result = result \cup \pi_{q_1,\ldots,q_m} R' \times \{s_1 \mapsto \perp, \ldots, s_k \mapsto \perp\}$
  else:
    $result = result \cup \{\{p_i \mapsto F_i(R') \mid i \in \{1, \ldots, n\}\}\}$
return *result*

The equivalence of this algorithm to the definition of the generalised group-by operator can be shown using a rewrite proof. In this rewrite proof we rewrite the algorithm into the set comprehension expressions that define the group-by operator. It is easy to see that this is possible, by observing that the 'if' part of the 'if' statement in the algorithm corresponds to the first set that constitutes the population of a grouped relation. The 'else' part corresponds to the second set that constitutes the population. A detailed proof is available online[4]. □

5 Imperfection Pattern Repair

This Section describes how the event log imperfection patterns [30] can be repaired using the (extended) relational algebraic operators introduced in the previous section. We only focus on those patterns where repair needs at least one of the dedicated operators (i.e., directly follows or generalised group-by). These are the Homonymous Labels, Collateral Events and Form-based Event Capture patterns. By convention, we label an input event log as R_i, and an output (repaired) event log as R_o. Any intermediate event logs that are created are labelled as $R_1, R_2, \cdots$.

[4] https://github.com/praeclaruspdq/PraeclarusPDQ/blob/master/Proof.pdf

cid	*act*	*time*	*res*
.	.	.	.
c_1	a_0	t_0	r_0
c_1	a_1	t_1	r_1
c_1	a_2	t_2	r_1
c_1	a_3	t_3	r_1
c_1	a_4	t_4	r_2
c_2	a_2	t_5	r_3
c_2	a_1	t_6	r_3
c_2	a_3	t_7	r_3
.	.	.	.

cid	*act*	*time*	*res*
.	.	.	.
c_1	a_0	t_0	r_0
$any(\{\|c_1, c_1, c_1\|\})$	a_{new}	$max(\{\|t_1, t_2, t_3\|\})$	$any(\{\|r_1, r_1, r_1\|\})$
c_1	a_4	t_4	r_2
$any(\{\|c_2, c_2, c_2\|\})$	a_{new}	$max(\{\|t_5, t_6, t_7\|\})$	$any(\{\|r_3, r_3, r_3\|\})$
.	.	.	.

Table 2: The generalised group-by operator is applied to the first table to aggregate activities from the set $\{a_1, a_2, a_3\}$ that belong to the same case and happen within a time δ of each other. The resulting groups are labelled a_{new} and have a completion time that is the last time of the original activities and a resource that is any of the resources of the original activities.

5.1 Homonymous Label

The *Homonymous Label* pattern refers to attribute values with the same syntax but different semantics. Figure 1 shows a personal injury compensation insurance log, where the activity label '*Review estimate*' has different meanings in different contexts. In this instance, the insurer's task management system was set to trigger a request for review of the estimate every time a new document was received. A request for review of the estimate could also be manually lodged by a supervisor. Finally, claim estimates were requested every 6 months for open claims. These periodic review requests were issued in batches very close in time to each other. Therefore, we have three different types of review requests, all recorded with label '*Review estimate*': (1) Automatic, issued by the system after the receipt of new documents; (2) Manual, issued by the supervisor; and (3) Periodic, issued by the system every 6 months for all open claims in batches.

To repair this pattern, we need to know the event that directly precedes each of the review estimate events, to see if they are triggered by the receipt of a new document. Applying the directly precedes operator returns R_1:

$R_1 = \pi_{\uparrow cid, \uparrow act, \uparrow time, \uparrow res, \downarrow act}(<_{cid,time}(R_i))$

R_1 is then updated to include all the events without a preceding event in the result (as the directly precedes operator would normally remove those events). The result is R_2 shown in Figure 2.

$R_2 = R_1 \cup \pi_{\uparrow cid, \uparrow act, \uparrow time, \uparrow res}(R_i \setminus (\pi_{\uparrow cid, \uparrow act, \uparrow time, \uparrow res}(R_1)))$

We then use the extended projection operator to update the instances of the *'Review estimate'* label based on the preceding label $\downarrow$ *act* and the resource attribute $\uparrow$ *res*. The function f updates any *'Review estimate'* activity that is not done by *'System'* to *'Manual review estimate'*, any *'Review estimate'* activity that is done by *'System'* and is not following a *'Document received'* activity by

cid	act	time	res
1	Document received	2202-01-31 11:30:45	100
1	Review estimate	2202-01-31 11:31:10	System
2	Document received	2202-01-31 13:15:30	100
2	Review estimate	2022-01-31 13:15:40	System
1	Review completed	2022-02-01 08:00:10	100
2	Review completed	2022-02-01 08:00:10	100
3	Document received	2202-04-11 08:30:45	100
3	Review estimate	2202-04-11 08:31:05	System
3	Review completed	2022-04-12 08:00:10	100
4	Review estimate	2202-04-12 09:15:30	200
4	Review completed	2202-04-19 11:25:30	300
1	Review estimate	2202-07-01 00:10:00	System
2	Review estimate	2202-07-01 00:10:01	System
3	Review estimate	2202-07-01 00:10:02	System

Fig. 1: The homonymous label *'Review estimate'* in an insurance log (R_i).

↑cid	↑act	↑time	↑res	↓act
1	Document received	2202-01-31 11:30:45	100	-
1	Review estimate	2202-01-31 11:31:10	System	Document received
2	Document received	2202-01-31 13:15:30	100	-
2	Review estimate	2022-01-31 13:15:40	System	Document received
1	Review completed	2022-02-01 08:00:10	100	Review estimate
2	Review completed	2022-02-01 08:00:10	100	Review estimate
3	Document received	2202-04-11 08:30:45	100	-
3	Review estimate	2202-04-11 08:31:05	System	Document received
3	Review completed	2022-04-12 08:00:10	100	Review estimate
4	Review estimate	2202-04-12 09:15:30	200	-
4	Review completed	2202-04-19 11:25:30	300	Review estimate
1	Review estimate	2202-07-01 00:10:00	System	Review completed
2	Review estimate	2202-07-01 00:10:01	System	Review completed
3	Review estimate	2202-07-01 00:10:02	System	Review completed

Fig. 2: Applying directly precedes operator on the log shown in Figure 1 (R_i). R_1 is then updated to include all the events without a preceding event (R_2).

'Periodic review estimate', and any *'Review estimate'* activity that is done by *'System'* and is following a *'Document received'* activity by *'Automatic review estimate'*. Applying the following operator on R_2 will repair the log (in Figure 3): $R_o = \pi_{cid:\uparrow cid, act:f(\uparrow act,\uparrow res,\downarrow act), time:\uparrow time, res:\uparrow res}(R_2)$, where with slight abuse of notation:

$$f=\{(\text{'Review est.'},x,y)\mapsto\text{'Manual review est.'}|x\in Dom(\downarrow res)\backslash\{\text{'System'}\}\wedge y\in Dom(\uparrow act)\}\cup$$
$$\{(\text{'Review est.'},\text{'System'},y)\mapsto\text{'Periodic review est.'}|y\in Dom(\uparrow act)\backslash\{\text{'Document received'}\}\}\cup$$
$$\{(\text{'Review est.'},\text{'System'},\text{'Document received'})\mapsto\text{'Automatic review estimate'}\}$$

5.2 Collateral Events

The *Collateral Events* pattern refers to a set of low-level events, happened within a short time frame, that are essentially one particular process step. A repair

cid	act	time	res
1	Document received	2202-01-31 11:30:45	100
1	**Automatic review estimate**	2202-01-31 11:31:10	System
2	Document received	2202-01-31 13:15:30	100
2	**Automatic review estimate**	2022-01-31 13:15:40	System
1	Review completed	2022-02-01 08:00:10	100
2	Review completed	2022-02-01 08:00:10	100
3	Document received	2202-04-11 08:30:45	100
3	**Automatic review estimate**	2202-04-11 08:31:05	System
3	Review completed	2022-04-12 08:00:10	100
4	**Manual review estimate**	2202-04-12 09:15:30	200
4	Review completed	2202-04-19 11:25:30	300
1	**Periodic review estimate**	2202-07-01 00:10:00	System
2	**Periodic review estimate**	2202-07-01 00:10:01	System
3	**Periodic review estimate**	2202-07-01 00:10:02	System

Fig. 3: Applying the extended projection operator on R_2 (shown in Figure 2) to repair homonymous labels.

cid	act	time
1	Submit claim	18/06/2014 03:20:09
1	Adjust recovery cost	19/06/2014 12:15:18
1	Email	19/06/2014 12:16:53
1	Pay assessor fee	19/06/2014 12:19:25
1	Approve claim	20/06/2014 8:40:01
2	Submit claim	20/06/2014 13:08:39
2	Email	21/06/2014 10:15:28
2	Adjust recovery cost	21/06/2014 10:16:43
2	Pay assessor fee	21/06/2014 10:19:53
2	Reject claim	22/06/2014 13:45:24

cid	act	time
1	Submit claim	18/06/2014 03:20:09
1	**Pay insurance claim assessor**	19/06/2014 12:15:18
1	Approve claim	20/06/2014 8:40:01
2	Submit claim	20/06/2014 13:08:39
2	**Pay insurance claim assessor**	21/06/2014 10:15:28
2	Reject claim	22/06/2014 13:45:24

Fig. 4: Collateral Events, before (R_i) and after (R_o) repair using the generalised group-by operator.

approach is to aggregate these events to a higher-level event using the generalised group-by operator $\mathcal{G}$. Figure 4 depicts an insurance event log where events with labels *'Adjust recovery cost'*, *'Email'*, and *'Pay assessor fee'* refer to the process step *'Pay insurance claim assessor'*. The following operator can be used to repair the log, as shown in Figure 4:

$R_o = {}_{GC(t_1,t_2)}\mathcal{G}_{cid \mapsto any_{cid}, act \mapsto anew, time \mapsto min_{time}} R_i$, where

$GC(t_1,t_2) = (t_1[cid] = t_2[cid]) \wedge t_1[act] \in \mathcal{L} \wedge t_2[act] \in \mathcal{L} \wedge |t_1[time] - t_2[time]| \leq \delta$

$\mathcal{L} = \{\text{'}Adjust\ recovery\ cost\text{'}, \text{'}Email\text{'}, \text{'}Pay\ assessor\ fee\text{'}\}$

$\delta = 600$

$anew(act) = \text{'}Pay\ insurance\ claim\ assessor\text{'}$

Note that for grouping events, the grouping condition $GC(t_1, t_2)$ usually has the same structure, in that we look for events from the same case ($t_1[cid] = t_2[cid]$)) that concern performing activities from a specific set ($t_1[act] \in \mathcal{L} \wedge t_2[act] \in \mathcal{L}$) within a specific timeframe of each other ($|t_1[time] - t_2[time]| \leq \delta$).

Consequently, one can simplify specifying the grouping condition by keeping its general formulation and only specifying $\mathcal{L}$ and δ.

cid	act	val	time
1	Triage		11/07/2015 04:20:09
1	Temperature	38	11/07/2015 06:12:18
1	Pulse	89	11/07/2015 06:12:18
1	Blood pressure	117	11/07/2015 06:12:18
1	Operation		11/07/2015 17:45:03
1	Temperature	37	11/07/2015 20:05:39
1	Pulse	81	11/07/2015 20:05:39
1	Discharge		12/07/2015 11:17:42

cid	act	temp	pulse	bp	time
1	Triage				11/07/2015 04:20:09
1	**Vital signs check**	38	89	117	11/07/2015 06:12:18
1	Operation				11/07/2015 17:45:03
1	**Vital signs check**	37	81		11/07/2015 20:05:39
1	Discharge				12/07/2015 11:17:42

Fig. 5: Form-based Event Capture, before (R_i) and after (R_o) repair using the generalised group-by operator

5.3 Form-based Event Capture

The *Form-based Event Capture* pattern refers to a set of low-level events that are essentially one particular process step. These events are captured through a form and have identical timestamps (the time that the form was "saved"). To repair this pattern, we can group these low-level events into a higher-level event using the generalised group-by operator $\mathcal{G}$. Figure 5 depicts a hospital event log where events with label *'Temperature'*, *'Pulse'*, and *'Blood pressure'* are recorded through an electronic form to check patient's vital signs. We can see that for the same patient (with $cid = 1$), this form has been recorded twice: the first time it records three vital signs *'Temperature'*, *'Pulse'*, and *'Blood pressure'* with their values, and the second time it records only the changed vital signs *'Temperature'* and *'Pulse'* (The blood pressure is not changed since the last check and therefore is not updated in the form and not recorded in the log). To repair this pattern, any subset of these three low-level events with the same timestamp is grouped into a single event labelled *'Vital signs check'*. The value of this event would have additional columns for the pairs of labels and values of the low level events, e.g., (*'Temperature'*, 38). The following operator can be used to repair the log, as shown in Figure 5:

$$R_o = {}_{GC(t_1,t_2)}\mathcal{G}_{\substack{cid\mapsto any_{cid},act\mapsto anew,temp\mapsto temp,\\ pulse\mapsto pulse,bp\mapsto bp,time\mapsto min_{time}}} R_i, \text{ where}$$

$$GC(t_1,t_2)=(t_1[cid]=t_2[cid])\wedge t_1[act]\in\mathcal{L}\wedge t_2[act]\in\mathcal{L}\wedge t_1[time]=t_2[time]$$

$$\mathcal{L}=\{\text{'Temperature'},\text{'Pulse'},\text{'Blood pressure'}\}$$

$$anew(act)=\text{'Vital signs check'}$$

$$temp(ps)=v,\ if\ (\text{'Temperature'},v)\in ps,\ else\ \perp$$

$pulse$ and bp are defined analogously.

6 Evaluation

We evaluate the dedicated repair language in terms of its suitability and computational efficiency. To this end, we compare dedicated expressions and vanilla expressions to repair common patterns of imperfection in event logs. The repair expressions that use the dedicated operators are given in the previous sections. For completeness, we first present the repair expressions that use only vanilla operators in Section 6.1. We use these pairs of repair statements to compare the suitability (Section 6.2), computational efficiency (Section 6.3), and runtime efficiency (Section 6.4) of repair using vanilla operators vs. dedicated operators.

6.1 Imperfection pattern repair using vanilla operators

In this section, we show how three common patterns of imperfection in event logs, i.e., Homonymous Labels, Collateral Events, and Form-based Event Capture, can be repaired using only vanilla operators in contrast to the repair statements that use dedicated operators presented in Section 5.

Homonymous Labels The example event log from Figure 1 can be repaired by the following sequence of statements using vanilla relational algebra operators.

$R_1 = \pi_{cid,\uparrow act:act,\uparrow time:time,\uparrow res:res}(R_i)$
$R_2 = \pi_{cid,\downarrow act:act,\downarrow time:time,\downarrow res:res}(R_i)$
$R_3 = \pi_{cid,\uparrow act,\uparrow time,\uparrow res,\downarrow act,\downarrow time}(\sigma_{\uparrow time > \downarrow time}(R_1 \bowtie R_2))$
$R_4 = R_3 \setminus (\pi_{Schema(R_3)}(\sigma_{time < \uparrow time \wedge time > \downarrow time}(R_3 \bowtie R_i)))$
$R_5 = \pi_{cid,act:\uparrow act,time:\uparrow time,res:\uparrow res,\downarrow act}(R_4)$
$R_6 = R_5 \cup \pi_{cid,act,time,res,\text{“”}}(R_i \setminus (\pi_{cid,act,time,res}(R_5)))$
$R_o = \pi_{cid,act,time,res}(\sigma_{act \neq \text{'Review estimate'}}(R_6)) \cup$
$\pi_{cid,act:\text{'Manual review estimate'},time,res}($
$\quad \sigma_{act=\text{'Review estimate'} \wedge res \neq \text{'System'}}(R_6)) \cup$
$\pi_{cid,act:\text{'Periodic review estimate'},time,res}($
$\quad \sigma_{act=\text{'Review estimate'} \wedge res=\text{'System'} \wedge \downarrow act \neq \text{'Document received'}}(R_6)) \cup$
$\pi_{cid,act:\text{'Automatic review estimate'},time,res}($
$\quad \sigma_{act=\text{'Review estimate'} \wedge res=\text{'System'} \wedge \downarrow act=\text{'Document received'}}(R_6))$

Collateral Events The example event log from Figure 4 can be repaired using the following vanilla statements.

$R_1 = \pi_{cid,act_1:act,time_1:time}(\sigma_{act=\text{'Adjust recovery cost'}}(R_i))$
$R_2 = \pi_{cid,act_2:act,time_2:time}(\sigma_{act=\text{'Email'}}(R_i))$
$R_3 = \pi_{cid,act_3:act,time_3:time}(\sigma_{act=\text{'Pay assessor fee'}}(R_i))$
$R_4 = \sigma_{|Max(time_1,time_2,time_3)-Min(time_1,time_2,time_3)|<\delta}(R_1 \bowtie R_2 \bowtie R_3)$
$R_5 = \pi_{cid,act:\text{'Pay insurance claim assessor'},time:f_{time}(time_1,time_2,time_3)}(R_4)$
$R_6 = \pi_{cid,act,time}(\sigma_{act=act_1 \wedge time=time_1}(R_i \bowtie R_4)) \cup$
$\quad \pi_{cid,act,time}(\sigma_{act=act_2 \wedge time=time_2}(R_i \bowtie R_4)) \cup$
$\quad \pi_{cid,act,time}(\sigma_{act=act_3 \wedge time=time_3}(R_i \bowtie R_4))$
$R_o = (R_i \setminus R_6) \cup R_5$

Form-based Event Capture The example event log from Figure 5 can be repaired using the following vanilla statements:

$R_1 = \pi_{cid,act_1:act,val_1:val,time_1:time}(\sigma_{act='Temprature'}(R_i))$
$R_2 = \pi_{cid,act_2:act,val_2:val,time_2:time}(\sigma_{act='Pulse'}(R_i))$
$R_3 = \pi_{cid,act_3:act,val_3:val,time_3:time}(\sigma_{act='Blood\ pressure'}(R_i))$
$R_4 = \sigma_{time_1=time_2=time_3}(R_1 \rtimes R_2 \rtimes R_3)$[5]
$R_5 = \pi_{cid,act:'Vital\ signs\ check',temp:val_1,pulse:val_2,bp:val_3,time:any(time_1,time_2,time_3)}(R_4)$
$R_6 = \pi_{cid,act,val,time}(\sigma_{act=act_1 \wedge val=val_1 \wedge time=time_1}(R_i \ltimes R_4)) \cup \pi_{cid,act,val,time}(\sigma_{act=act_2 \wedge val=val_2 \wedge time=time_2}(R_i \ltimes R_4)) \cup \pi_{cid,act,val,time}(\sigma_{act=act_3 \wedge val=val_3 \wedge time=time_3}(R_i \ltimes R_4))$
$R_o = (\pi_{cid,act,temp,pulse,bp,time}(R_i \setminus R_6)) \cup R_5$

6.2 Suitability

When specifying event log repair expressions, it is important to maintain simplicity. If, e.g., there are too many operators or parameters, or if the nesting depth of the statements is too high, it will be hard for the user to understand and track the changes made to the original log in the repair process. This also increases the probability of making errors in developing such statements. Therefore, for each event log imperfection pattern, we compare the suitability of the repair statements that use only the vanilla database operators with the repair statements that use the dedicated operators.

Table 3 compares the suitability of the vanilla and dedicated repair statements for each of the three aforementioned imperfection pattern. We used the Halstead metrics [17] for this purpose. These metrics represent the number of unique operators (η_1) and operands (η_2), their total occurrences (N_1 and N_2), the expression vocabulary ($\eta = \eta_1 + \eta_2$), length ($N = N_1 + N_2$), volume ($V = N \log_2 \eta$), difficulty ($D = (\eta_1/2)(N_2/\eta_2)$), and effort ($E = D \cdot V$). These metrics are well-established, and there are known relations between the complexity as measured by these metrics and, for example, the probability that there is a bug. All metrics show that the expressions that use the dedicated operators are about half as complex as the expressions that use the vanilla operators.

Table 3: Suitability analysis of pattern repair with dedicated vs. vanilla operators

Case	**Database**	η_1	η_2	N_1	N_2	η	N	V	D	E
Homonymous labels	Dedicated	14	38	34	83	52	117	667	15	10197
	Vanilla	13	33	65	126	46	191	1055	25	26183
Collateral events	Dedicated	11	26	27	51	37	78	406	11	4384
	Vanilla	12	34	54	93	46	147	812	16	13326
Form-based event capture	Dedicated	14	34	41	66	48	107	598	14	8120
	Vanilla	14	32	73	109	46	182	1005	24	23970

[5] $R \rtimes R'$ is the join including unmatched tuples or R padded with null values

6.3 Theoretical Computational Complexity

We measure the computational complexity in terms of the number of basic functions that are applied, including simple operations, such as addition and multiplication, and also user-defined operations, such as the generalized group by operator. This is an approximation and in actuality the computational complexity may vary depending on the specific user-defined operations that are used. However, the user-defined operations are the same both in the dedicated database operators and in the vanilla database operators, such that their (number of) applications can be compared.

The computational complexity of the directly follows and the directly precedes relation is $O(N^2)$ [11]. The computational complexity of the generalised group-by operator is $O(N^2/2 + 2N)$, where $N = |R|$ is the number of rows in the table R. This complexity follows from the following implementation of the operator. We iterate over all rows. We store each row into a buffer and then iterate over all subsequent rows to check if they can be grouped with rows already in the buffer, storing rows that can be grouped in the buffer and removing them from the table. After comparing the row to all subsequent rows, we group the rows in the buffer, aggregate the result, and clear the buffer. Comparing each row to subsequent rows takes $O(N^2/2)$, applying the grouping functions to each row takes N steps and storing the result takes N steps.

Considering the approximate computational complexity of the operators, Table 4 presents the computational complexity of the repair statements both using dedicated (directly follows and generalised group-by) operators and using only vanilla operators. In these formulas $N = |R|$ is the number of rows, H is the number of homonyms that must be corrected, M the number of activity labels that must be corrected, and K the total number of activity labels.

Table 4: Computational complexity of repair patterns

Repair pattern	**Complexity (dedicated)**	**Complexity (vanilla)**
Homonymous Label	$O(2N^2 + 7N)$	$O(6N^2 + 2HN)$
Collateral Events	$O(N^2/2 + 2N)$	$O(MN + 2(N/K)^M + N\frac{M}{K}N)$
Form-based Event Capture	$O(N^2/2 + 2N)$	$O(MN + 2(N/K)^M + N\frac{M}{K}N)$

The computational complexity formulas follow directly from the way in which the respective repairs are performed. For example, the repair of homonymous labels with the dedicated engine has $O(3N^2 + 4N)$ complexity, because it requires three passes over the table with different operators. The first pass applies the directly precedes operator and projection, the second pass applies projection, set minus, projection and set union, and the third pass applies extended projection. projection has complexity $O(N^2)$, set minus and union have complexity $O(2N)$ (worst case), and directly follows and extended projection have complexity $O(N^2)$. The other complexity characterisations are derived in a similar way.

Overall, this comparison shows that repairing event log imperfection patterns using dedicated database operators can be done with a lower computational complexity compared to using only vanilla database operators for repair.

6.4 Runtime Efficiency

Runtime efficiency is measured as the number of milliseconds required to repair an event log containing imperfection patterns. The dedicated operators, as well as the vanilla operators, are implemented in PraeclarusPDQ[6], an open-source framework for process-data quality management. We also composed the SQL equivalent of vanilla repair expressions to compare the runtime efficiency. The SQL was implemented in an installation of MS SQL Server Community Edition. The experiments were conducted on a Core i7 processor and 16GB RAM system.

We conducted experiments using a real-world event log from an Australian hospital that describes the care process for patients presenting to the Emergency Department (ED) with chest pain[7]. The log contains 15,135 events comprising 38 distinct activity labels relating to 293 cases (ED presentations. Where an activity relates to some form of clinical observation, the observation value is recorded as an event attribute. The log contains examples of the Form-based Event Capture pattern representing two forms frequently completed during an ED presentation. The '*Vital Signs Check*' form (F1) is completed periodically during the presentation to monitor the patient's well-being, while the '*Triage*' form (F2) is completed by clinical staff in the early stage of a presentation to assess and prioritise the patient's treatment. The '*Vital Signs Check*' form comprises up to 5 activities in the log: '*Oxygen Saturation*', '*Pulse*', '*Respiratory Rate*', '*BP Systolic Lying*', and *'BP Diastolic Lying*', while the '*Triage*' form comprises up to 8 activities: '*AirwayClear*', '*BreathingDepth*', '*BreathingQuality*', '*BreathingRate*', '*CirculationPulse*', '*CirculationRhythm*', '*CirculationVolume*', and '*PainScale-Numeric*'. We found 1,113 instances of the '*Vital Signs Check*' form, and 293 instances of the '*Triage*' form in the log.

Table 5 reports the time performance for various approaches, i.e., our dedicated operator (generalised group-by), vanilla relational algebra, and SQL queries, to repair occurrences of F1 and F2. For each approach, we ran three experiments to (1) repair F1 in the input log, (2) repair F2 in the input log, and (3) repair F2 in the log after having already repaired F1. This log has fewer events (11,082) compared to the input log (15,135). We can see that the dedicated operator repair takes less time in the third experiment, where the number of events or instances of the form is lower than in the first two experiments. The results further show that repairing F1 using the dedicated operator, which takes 4,037 ms, is 86 times more efficient than the vanilla operator repair, which takes 350,170 ms. The same behaviour is observed for the next two experiments, where the dedicated operator runs 50 and 59 times faster than the vanilla operators. The runtime of SQL queries lies between that of the dedicated operator and vanilla operators, and is comparable only to the dedicated operator in the first experiment.

[6] Available at `https://github.com/praeclaruspdq/PraeclarusPDQ`.
[7] We cannot release this log due to the NDA agreement entered into with the hospital.

There are a number of factors that may impact on the running time of any method. These include the size of the log, the complexity of each form in terms of the number of activities included in each form, the number of instances of each form in the log, and the completeness of each form (the number of times all of the activities comprising the form are present in any instance of the form). In this example, F1 is 92% complete while F2 is 98% complete. These factors are related to the input data and are the same for all methods. Additional impacts arise from hardware (again same for all methods tried here) and the environment in which the method is implemented. For instance, the SQL queries were executed in a DBMS that develops a statistically based execution plan to ensure that the database engine uses the most efficient path to execute a query. The relative effect of each of these factors on individual repair methods has yet to be investigated.

Table 5: Running time (ms) of Form-based Event Capture repair in a real log

Form to repair	**#instances**	**#input events**	**Time (ms) - dedicated**	**Time (ms) - vanilla**	**Time (ms) - SQL**
Repairing F1 only	1,113	15,135	4,073	350,170	4,888
Repairing F2 only	289	15,135	721	35,838	13,693
Repairing F2 after F1	289	11,082	434	25,515	12,569

7 Conclusion

In this paper, we introduced a language for expressing event log repair operations. To this end, we introduced the generalised group-by operator, the second process mining-specific operator after the directly follows operator. The operator was defined in relational algebra and implemented together with other dedicated operators in PraeclarusPDQ. We showcased event log imperfection patterns, the repair of which can be expressed using our language, extended by the two process mining-specific operators. We evaluated the suitability, computational efficiency, and runtime performance of repairing event logs using the new operators versus using vanilla (SQL) database operators.

The results show that the use of the generalised group-by and the directly follows operators yields more suitable event log repair expressions (i.e. fewer operators, fewer parameters, and reduced nested operators' depth). Furthermore, our analysis shows that the computational efficiency is also improved as a result of using these dedicated database operators. An evaluation using our prototype platform for event log repair shows that the runtime performance of dedicated repair expressions is at least 50 times higher on a real-world event log, compared to expressions formulated with standard database operators. Future work can explore adapting the generalised group-by operator to repair data quality issues in other types of datasets, such as object-centric and IoT event logs.

References

1. van der Aalst, W.M.: Process Mining - Data Science in Action. Springer (2016)
2. Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Int. Conf. Manag. Data. pp. 207–216. ACM, Washington D.C. (1993)
3. van Bommel, P., ter Hofstede, A.H., van der Weide, T.: Semantics and verification of object-role models. Inf. Syst. **16**(5), 471–495 (1991)
4. Cheng, H.J., Kumar, A.: Process mining on noisy logs—can log sanitization help to improve performance? Decis. Support Syst. **79**, 138–149 (2015)
5. Chien, M., Jain, A.: Magic quadrant for data quality tools. Gartner (2019)
6. Codd, E.F.: A relational model of data for large shared data banks. Communications of the ACM **13**(6), 377–387 (1970)
7. Conforti, R., La Rosa, M., ter Hofstede, A.: Filtering out infrequent behavior from business process event logs. IEEE Trans. Knowl. Data Eng. **29**(2), 300–314 (2016)
8. Conforti, R., La Rosa, M., ter Hofstede, A., Augusto, A.: Automatic repair of same-timestamp errors in business process event logs. In: BPM. pp. 327–345. Springer, Seville, Spain (2020)
9. Côté, P.O., Nikanjam, A., Ahmed, N., Humeniuk, D., Khomh, F.: Data cleaning and machine learning: a systematic literature review. Automated Software Engineering **31**(2), 54 (2024)
10. Dayal, U., Goodman, N., Katz, R.H.: An extended relational algebra with control over duplicate elimination. In: Symp. Principles of Database Syst. pp. 117–123. ACM, LA, USA (1982)
11. Dijkman, R., Gao, J., Syamsiyah, A., van Dongen, B.F., Grefen, P., ter Hofstede, A.H.: Enabling efficient process mining on large data sets: Realizing an in-database process mining operator. Distrib. Parallel Databases **38**(1), 227–253 (2020)
12. Dixit, P.M., et al.: Detection and interactive repair of event ordering imperfection in process logs. In: CAiSE. pp. 274–290. Springer, Tallinn, Estonia (2018)
13. van Dongen, B.F., Shabani, S.: Relational XES: data management for process mining. In: CAiSE Forum. CEUR, vol. 1367, pp. 169–176. Sweden (2015)
14. Ehrlinger, L., Wöß, W.: A survey of data quality measurement and monitoring tools. Frontiers in big data **5**, 850611 (2022)
15. Gray, J., Bosworth, A., Layman, A., Piranesh, H.: Data Cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. In: ICDE. pp. 560–573. IEEE, NO, USA (1996)
16. Grefen, P., de By, R.: A multi-set extended relational algebra: A formal approach to a practical issue. In: Int. Conf. Data Eng. pp. 80–88. IEEE, USA (1994)
17. Halstead, M.H.: Elements of Software Science. Elsevier (1977)
18. Horng, J.T., Chen, G.D., Liu, B.J.: An extension of sql to capture more information from objects with many-many relationship. In: TENCON. pp. 356–359. IEEE, China (1993)
19. Khedri, R., Chiang, F., Sabri, K.E.: An algebraic approach towards data cleaning. Procedia Computer Science **21**, 50–59 (2013)
20. Klug, A.: Equivalence of relational algebra and relational calculus query languages having aggregate functions. Jour. of the ACM **29**(3), 699–717 (1982)
21. Kohlas, J., Stärk, R.F.: Information algebras and consequence operators. Logica Universalis **1**, 139–165 (2007)
22. Maier, D.: The theory of relational databases. Computer Science Press (1983)

23. Meo, R., Psaila, G., Ceri, S.: An extension to sql for mining association rules. Data Min. Knowl. Discov. **2**, 195–224 (1998)
24. Neutatz, F., Chen, B., Abedjan, Z., Wu, E.: From Cleaning before ML to Cleaning for ML. IEEE Data Eng. Bull. **44**(1), 24–41 (2021)
25. Núnez-Corrales, S., Li, L., Ludäscher, B.: A first-principles algebraic approach to data transformations in data cleaning: Understanding provenance from the ground up. In: TaPP. pp. 1–11. USENIX Assoc., Philadelphia, USA (2020)
26. Sadeghianasl, S., ter Hofstede, A.H.M., Wynn, M.T., Türkay, S.: Humans-in-the-loop: Gamifying activity label repair in process event logs. Eng. Appl. Artif. Intell. **132**, 107875 (2024)
27. Sadeghianasl, S., ter Hofstede, A.H., Suriadi, S., Turkay, S.: Collaborative and interactive detection and repair of activity labels in process event logs. In: ICPM. pp. 41–48. IEEE, Italy (2020)
28. Sadeghianasl, S., Wynn, M.T., Andrews, R., van der Aalst, W.M.P., Ko, J.: Object-centric event-data imperfection patterns. Inf. Syst. **141**, 102766 (2026). https://doi.org/10.1016/J.IS.2026.102766, `https://doi.org/10.1016/j.is.2026.102766`
29. Sadeghianasl, S., et al.: A contextual approach to detecting synonymous and polluted activity labels in process event logs. In: CoopIS. pp. 76–94. Greece (2019)
30. Suriadi, S., Andrews, R., ter Hofstede, A.H., Wynn, M.T.: Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. Inf. Syst. **64**, 132–150 (2017)
31. Viqueira, J.R.R., Lorentzos, N.A.: Sql extension for spatio-temporal data. The VLDB Journal **16**, 179–200 (2007)
32. Walsh, N., Snelson, J., Coleman, A.: Xquery and xpath data model 3.1 (2021), `https://www.w3.org/TR/xpath-datamodel/`
33. Weis, M., Manolescu, I.: Declarative XML data cleaning with XClean. In: CAiSE. pp. 96–110. Springer, Trondheim, Norway (2007)
34. Whang, S.E., Roh, Y., Song, H., Lee, J.G.: Data collection and quality challenges in deep learning: A data-centric ai perspective. The VLDB Journal **32**(4), 791–813 (2023)
35. Wynn, M.T., Lebherz, J., van der Aalst, W.M., Accorsi, R., Di Ciccio, C., Jayarathna, L., Verbeek, H.: Rethinking the input for process mining: Insights from the XES Survey and Workshop. Tech. rep., IEEE Task Force on Process Mining (2021)
36. Yun, J., Ahn, H., Kim, K.P.: Tailoring operations based on relational algebra for xes-based workflow event logs. J. Internet Computing and Services **20**(6), 21–28 (2019)